

DATALINK



数据链

DATALINK

该项目用于不同数据源之间的多种数据同步方式.

结构

项目包含两部分:linkd组件和loop组件.loopadmin提供任务管理页面

linkd:提供web接口,用于管理task查看task运行状态

loop:用于运行task和pipeline处理

loopadmin:web页面

运行

```
# 下载仓库
git clone https://gitee.com/yz_rd_basic_services/data-link-2.0.git
# 构建bin
cd ./app/datalink
go build -o datalink2 main.go
# 运行项目
cd ../../
cp example.config.toml config.toml
./app/datalink/datalink2 -f config.toml
```

任务管理

一个完整的任务执行过程为:

1. 创建任务
2. 启动任务
3. 停止任务

4. 删除任务

HTTP接口

提供管理任务的接口

WEB页面

http://localhost:19191/debug_ui

hello world

```
GET http://localhost:19191/index
```

查看所有task

```
GET http://localhost:19191/tasks
=>
{
  "code": "10000",
  "data": [
    {
      "Desc": "mongodb to mysql use replica",
      "FailedDeliveryCount": 0,
      "FailedPipeLineCount": 0,
      "FullDeliveryCount": 0,
      "FullReadCount": 0,
      "Id": "78b75e14-3e20-11ec-84f9-0a002700000a",
      "LatestRunDuration": "0s",
      "LatestRunEndAt": 0,
      "LatestRunError": "2021-11-05 19:13:45.6255166 +0800 CST
m=+8.950459501 start message chan ~ ",
      "LatestRunStartAt": 0,
      "ReadDone": false,
      "State": "null",
      "SyncMode": "replica",
      "WriteDone": false
    }
  ],
  "msg": "success"
}
```

查看task完整错误

```
GET http://localhost:19191/task_error?id=d33752fe-3d1a-11ec-9481-0a002700000a
```

创建一个**task**,不会立即运行

```
PUT http://localhost:19191/task
<=
{
  "setup": {
    "desc": "mongodb to mysql use replica",
    "error_record": true
  },
  "resource": [
    {
      "id": "no1000001",
      "desc": "mongodb本地服务",
      "type": "mongodb",
      "user": "admin",
      "pass": "94215b0cb86d9ceb",
      "host": "47.114.84.103",
      "port": "33017",
      "dsn": "mongodb://admin:94215b0cb86d9ceb@47.114.84.103:33017/?
connect=direct"
    },
    {
      "id": "no1000002",
      "desc": "mysql本地服务",
      "type": "mysql",
      "user": "root",
      "pass": "root",
      "host": "127.0.0.1",
      "port": "3306",
      "dsn": ""
    }
  ],
  "source": [
    {
      "resource_id": "no1000001",
      "sync_mode": "replica",
      "document_set": "datalink_foo.area",
      "extra": {
        "resume": true,
        "watch_event": [
          "insert",
          "update",
          "replace",
          "delete"
        ]
      }
    }
  ],
  "target": [
    {
      "resource_id": "no1000002",
      "document_set": {
```

```
    "datalink_foo.area": "test.area_stream"
  },
  "extra": {
    "flush_interval": 10
  }
},
"pipeline": []
}
```

启动一个**task**,开始同步数据

```
POST http://localhost:19191/task
<=
{
  "id": "78b75e14-3e20-11ec-84f9-0a002700000a",
  "op": "start"
}
```

停止一个**task**

```
POST http://localhost:19191/task
<=
{
  "id": "8b44d60e-3868-11ec-b127-0a002700000a",
  "op": "stop"
}
```

删除一个**task**,先要停止**task**

```
DELETE http://localhost:19191/task
<=
{
  "id": "f1ba5ad0-3607-11ec-a14d-0a002700000a"
}
```

命令行接口(仅在**linux**下支持)

```
# 从文件中创建任务
datalink -new file_name.json

# 启动一个任务
```

```
datalink -start 7861237098102381238

# 停止一个任务
datalink -stop 7861237098102381238

# 移除一个任务
datalink -remove 7861237098102381238

# 任务信息
datalink -info 7861237098102381238

# 任务列表
datalink -list
```

task 介绍

所有任务都是工作在组件loop之中,loop组件管理所有的task的生成和销毁

使用

编辑一个task文件,将task文件通过web接口,提交至loop中.

完整的配置文件项

示例:

```
{
  // 程序相关设置
  "setup": {
    // 描述
    "desc": "from mongodb to elasticsearch",
    // 是否记录错误.可以通过web接口task_error,查看具体的task日志
    "error_record": true
  },
  // 资源,用于建立连接,source的读取,target写入,pipeline中relate的数据源
  "resource": [
    {
      // 资源id,resource唯一
      // dsn 和 (user pass host port) 仅选填一项.同时填写时,优先使用dsn.
      // rabbitmq比较特殊,仅支持dsn,因为需要指定dsn
      "id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
      "desc": "mongodb服务器",
      // 资源类型:mysql mongodb elasticsearch plaintext kafka rabbitmq
      "type": "mongodb",
      "user": "",
      "pass": "",
      "host": "",
      "port": "",
      // 优先使用 dsn
      "dsn": "mongodb://admin:94215b0cb86d9ceb@47.114.84.103:33017/?connect=direct"
```

```

    },
    {
      "id": "c888b053-45f3-4c98-b219-8acfe67999d1",
      "type": "mysql",
      "user": "root",
      "pass": "root",
      "host": "127.0.0.1",
      "port": "3306",
      "dsn": ""
    },
    {
      "id": "bf907d93-e499-449f-92f2-cd2f057450bc",
      "desc": "elasticsearch服务器",
      "type": "elasticsearch",
      "user": "elastic",
      "pass": "Ie#5D8FG@4fC7gJ9y+",
      "host": "http://db-01.yuzhua-test.com",
      "port": "43092",
      "dsn": ""
    }
  ],
  // 数据来源
  "source": [
    {
      // 需要用到的 resource,只支持单数据源读取
      "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
      // 读取方式:direct dump stream replica empty
      "sync_mode": "direct",
      // 需要处理的文档集
      "document_set": "database.document_set",
      // 额外参数,针对于source设置
      "extra": {
        // 是否继续,仅当资源类型为MySQL,MongoDB,同步模式为replica,stream时有
        "resume": true,
        // 仅当资源类型为MySQL,同步模式为replica,stream时有效
        "onDDL": true,
        // 每次读取数量,不同数据源读取数据的方式,一般情况下为批量取值限制
        // 可根据每行数据大小合理调整,防止出现写入错误
        "limit": 100
      }
    },
    {
      // 引用的资源id,将资源id作为
      "resource_id": "c888b053-45f3-4c98-b219-8acfe67999d1",
      // 用于建立连接
      "sync_mode": "empty",
      "document_set": "",
      "extra": {
      }
    }
  ],
  // 写入
  "target": [

```

效

```

    {
      // resource id
      "resource_id": "c888b053-45f3-4c98-b219-8acfe67999d1",
      // 文档集映射,
      "document_set": {
        // source的文档集名称:目标的文档集名称
        "database.document_set": "document_set_index001"
      },
      // 额外参数
      "extra": {
      }
    }
  ],
  //文档处理管道
  "pipeline": [
    {
      // 需要处理的文档集,从source中处理
      "document_set": "database.document_set",
      // 处理单元
      "flow": [
        {
          // 处理单元类型
          "type": "map",
          // 处理单元的脚本内容,使用的JSON文档不支持换行等
          "script": "module.exports=function(doc){doc.attr1=2022;return
doc}"
        },
        {
          // 处理单元为 filter,过滤文档
          "type": "filter",
          // 返回true即为忽略文档
          "script": "module.exports=function(doc){return false;}"
        },
        {
          // 使用字段映射,内置规则比map效率高.只是变化字段名建议该字段
          "type": "mapInline",
          "field_map": [
            {
              "srcField": "id",
              "srcType": "long",
              "aimField": "tab_id",
              "aimType": "long"
            },
            {
              "srcField": "code",
              "srcType": "string",
              "aimField": "tab_code",
              "aimType": "string"
            },
            {
              "srcField": "name",
              "srcType": "string",
              "aimField": "tab_name",
              "aimType": "string"
            }
          ]
        }
      ]
    }
  ]
}

```

```

    }
  ]
},
{
  // 处理单元为 relateInline,从另外的数据源中获取数据
  "type": "relateInline",
  // 关联关系
  "assoc_type": "11",
  // 脚本,预留,暂不支持
  "script": "",
  // 关联资源源resource id,relate_resource_id 更好
  "relate_source_id": "c888b053-45f3-4c98-b219-8acfe67999d1",
  // 关联数据集
  "relate_document_set": "database2.document_set1",
  // 层级关系:sib同级 sub子级
  "layer_type": "sib",
  // 子级时设置的键名称
  "sub_label": "",
  // 同级时,字段映射
  "field_map": {
    "r.id": "area_desc_id"
  },
  // 关联字段
  "wheres": [
    {
      // 源字段
      "src_field": "id",
      // 无效,默认是 相等
      "operator": "=",
      // 目标字段
      "rel_field": "id"
    }
  ]
}
]
}
]
}
}

```

resource中,当type=MongoDB时,使用dsn密码部分需要urlencode resource中,当type=Elasticsearch时,使用dsn示例:http://[user]:[pass]@host:port/ resource中,当type=RabbitMQ时,使用dsn部分需要vhost.示例:amqp://auto_brand-rabbitmq:password@1.1.1.1:5672/datalink

数据源

数据源的功能主要是不同方式的读取,以及作为关联数据源取数据

read_mode/source_type	MySQL	MongoDB	Elasticsearch	Plaintext	Kafka	RabbitMQ	--- --- --- ---
direct(完整同步)	√	√	√	√	×	×	stream(流同步)
stream(流同步)	√	√	×	×	√	√	replica(副本同步)
replica(副本同步)	√	√	×	×	×	×	empty(只是建立连接)
empty(只是建立连接)	√	√	√	√	×	×	

不同的数据源,限制使用.Plaintext 建立empty是无效的,不具备关联条件

目标源

目标源的:写入端的功能,主要实现三个功能,使用不同的操作将数据写入目标源

| write/target_type | MySQL | MongoDB | Elasticsearch | Plaintext | Kafka | RabbitMQ | | --- | --- | --- | --- | --- | |
insert | √ | √ | √ | √ | √ | √ | | update | √ | √ | √ | × | √× | √× | | delete | √ | √ | √ | × | √× | √× |

√×为变形支持,当在extra参数中设置format:true时,会变换消息格式.

pipeline 的处理器

实现对于文档结构编辑,文档字段的修改,删除等.处理关联关系等

map

文档字段处理,使用JavaScript脚本对文档字段编辑操作

```
{
  "type": "map",
  "script": "module.exports=function(doc){doc.attr1=2022;return doc}"
}
```

filter

是否忽略掉文档,使用JavaScript脚本忽略不满足条件的文档

```
{
  "type": "filter",
  "script": "module.exports=function(doc){return false;}"
}
```

mapInline

字段映射,用于映射源数据和目标数据的字段名和,字段类型修改.

目前字段类型只支持 number->string

```
{
  "type": "mapInline",
  "field_map": [
    {
      "srcField": "id",
      "srcType": "long",
      "aimField": "tab_id",
      "aimType": "long"
    }
  ]
}
```

```

    },
    {
      "srcField": "code",
      "srcType": "string",
      "aimField": "tab_code",
      "aimType": "string"
    },
    {
      "srcField": "name",
      "srcType": "string",
      "aimField": "tab_name",
      "aimType": "string"
    }
  ]
}

```

relateInline

关联操作,查询不同数据源的数据,进行数据关联

```

{
  "type": "relateInline",
  "assoc_type": "11",
  "source_id": "c888b053-45f3-4c98-b219-8acfe67999d1",
  "relate_document_set": "datalink_foo.area_desc",
  "layer_type": "sib",
  "sub_label": "",
  "field_map": {
    "r.id": "area_desc_id"
  },
  "wheres": [
    {
      "src_field": "id",
      "operator": "=",
      "rel_field": "id"
    }
  ]
}

```

source 介绍

数据读取来源,目前支持MySQL,MongoDB,Elasticsearch,Kafka,RabbitMQ

示例:

```

{
  "source": [
    {
      // 需要用到的 resource,只支持单数据源读取
    }
  ]
}

```

```
    "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
    // 读取方式
    "sync_mode": "direct",
    // 需要处理的文档集
    "document_set": "database.document_set",
    // 额外参数,针对于resource设置
    "extra": {
    }
  }
]
```

目前只支持单数据源同步:
resource_id为指定的资源id,即数据读取来源. sync_mode同步模式
document_set数据集
extra额外参数,比如MySQL的resume,limit,onDDL.

read_mode/source_type	MySQL	MongoDB	Elasticsearch	Plaintext	RabbitMQ	Kafka
direct(完整同步)	√	√	√	√	x	x
dump(完整同步)	x	√	x	x	x	x
stream(流同步)	√	√	x	x	√	√
replica(副本同步)	√	√	x	x	x	√
empty(只是建立连接)	√	√	√	√	x	x

- direct 完整的数据表同步
- dump 完整的数据表同步,依赖于数据源提供的具体能力.例如mysqldump
- stream 流同步,从运行任务开始时间点作为起点,开始同步数据
- replica 副本同步,会先执行完整数据表同步,在执行stream同步,保证完整的数据备份
- empty 不做任何动作,有时候是为了提供查询的通道

各数据源参数说明

MySQL

- 支持的同步模式: direct dump stream replica empty
- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "sync_mode": "direct",
  "document_set": "test.country",
  "extra": {
    // 支持参数
    "limit": 100,
    "resume": true,
    "onDDL": true,
  }
}
```

```

    "read_type": "page"
    // stream
  }
}

```

- read_type 读取数据的方式, - **stream**(默认)一次查询返回所有结果 - **page**分页换回数据
- limit 当**reader_type=page**时,有效.
- resume 是否继续当前任务进度,仅包含**stream**和**replica**时有效.默认**false**
- onDDL 是否同步修改表结构的语句,默认**false**

MongoDB

- 支持的同步模式: **direct dump stream replica empty**
- 作为source支持的extra参数:

```

{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "sync_mode": "direct",
  "document_set": "test.country",
  "extra": {
    // 支持参数
    "resume": true,
    "direct_resume": true,
    "watch_event": [
      "insert",
      "update",
      "delete"
    ]
  }
}

```

- resume 是否继续当前任务进度,仅当**stream**和**replica**时有效.默认**false**
- direct_resume 是否继续当前任务进度,仅当**direct**时有效.默认**false**
- watch_event 观察MongoDB的事件,(**insert,update,replace,delete** 默认值) 当**stream**和**replica**有效

Elasticsearch

- 支持的同步模式: **direct**
- 作为source支持的extra参数:

```

{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "sync_mode": "direct",
  "document_set": "test.country",
  "extra": {
    // 支持参数
    "limit": 100
  }
}

```

```
}
}
```

- limit 每次读取文档量(100 默认)

RabbitMQ

- 支持的同步模式: **stream**
- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "sync_mode": "direct",
  "document_set": "test.country",
  "extra": {
    // 支持参数
    "queue_name": "",
    "routing_key": "",
    "exchange": "",
    "pk_field": ""
  }
}
```

- queue_name 消息队列名称
- routing_key 路由键名称
- exchange 交换机名称
- pk_field 从读取的文档中指定那个字段作为文档主键

目前单任务只支持读取单个订阅

Kafka

- 支持的同步模式: **stream,replica**
- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "sync_mode": "direct",
  "document_set": "test.country",
  "extra": {
    // 支持参数
    "resume": true,
    "pk_field": "field_name"
  }
}
```

- resume 是否继续

- `pk_field` 从读取的文档中指定那个字段作为文档主键

因为kafka本身的特性,支持数据replay,所以resume的功能也是基于kafka本身保留的数据

特别说明:

- RabbitMQ和Kafka的消息只支持JSON格式字符串,其他格式字符格式化会错误.
- 所有的读取数据记录都有文档id的概念
- MongoDB同步至MySQL中,会将_id,作为docID
- MongoDB同步至Elasticsearch中,会将_id,作为docID

target 介绍

数据读取来源,目前支持MySQL,MongoDB,Elasticsearch,Kafka,RabbitMQ

示例:

```
{
  "target": [
    {
      // 资源id
      "resource_id": "bf907d93-e499-449f-92f2-cd2f057450bc",
      // 数据集映射关系
      "document_set": {
        // 源数据 => 目标数据
        "database.source_set": "database.target_set"
      },
      // 额外参数
      "extra": {
      }
    }
  ]
}
```

- `resource_id`资源ID,用于建立连接,构建对象
- `document_set`文本集映射,将数据源的文本集写入目标源的文本集
- `extra`额外参数,比如Elasticsearch一次刷写的数据量

MySQL

无特别说明

MongoDB

无特别说明

Elasticsearch

- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "document_set": {
    "database.source_set": "database.target_set"
  },
  "extra": {
    // 支持参数
    "limit": 100
  }
}
```

- limit 每次刷写文档量,设置limit参数即代表开启了批量写文档,默认是一条一条写入文档.建议设置,开启能极大提升写入效果

RabbitMQ

- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "document_set": {
    "database.source_set": "database.target_set"
  },
  "extra": {
    // 支持参数
    "routing_key": "routing_key",
    "exchange": "exchange",
    "format": true
  }
}
```

- routing_key 路由键名称
- exchange 交换机名称
- format 是否格式文档,文档本身是具有insert,update,delete属性的,当format=true时会将消息转换为:
 - {"OptionType": "insert", "data": json_data}

Kafka

- 作为source支持的extra参数:

```
{
  "resource_id": "a9e96c08-56de-4d97-a504-59ddaccee8c6",
  "document_set": {
    // 数据集名称 => kafka的topic
    "database.source_set": "database.target_set"
  },
  "extra": {
    // 支持参数
  }
}
```

```

    "format": true
  }
}

```

- `format` 是否格式文档,文档本身是具有`insert,update,delete`属性的,当`format=true`时会将消息转换为:
 - `{"OptionType": "insert", "data": json_data}`

在kafka中,映射的数据集中为`topic`

其他

- `document_set` 说明
 - `resource_id`为MongoDB时,`document_set`需要为`db.collection`;
 - `resource_id`为MySQL时,`document_set`需要为`db.table`;
 - `resource_id`为ES时,`document_set`需要为`index`;
 - `resource_id`为plaintext时,无需指定

pipeline 介绍

对于文档的提供删除字段,忽略字段,变更字段名称的功能

示例配置:

```

{
  "pipeline": [
    {
      "document_set": "database.document_set",
      "flow": [
        {
          "type": "map",
          "script": "module.exports=function(doc){doc.attr1=2022;return
doc}"
        },
        {
          "type": "filter",
          "script": "module.exports=function(doc){return false;}"
        },
        {
          "type": "mapInline",
          "field_map": [
            {
              "srcField": "id",
              "srcType": "long",
              "aimField": "tab_id",
              "aimType": "srcType"
            },
            {
              "srcField": "code",
              "srcType": "string",
              "aimField": "tab_code",

```

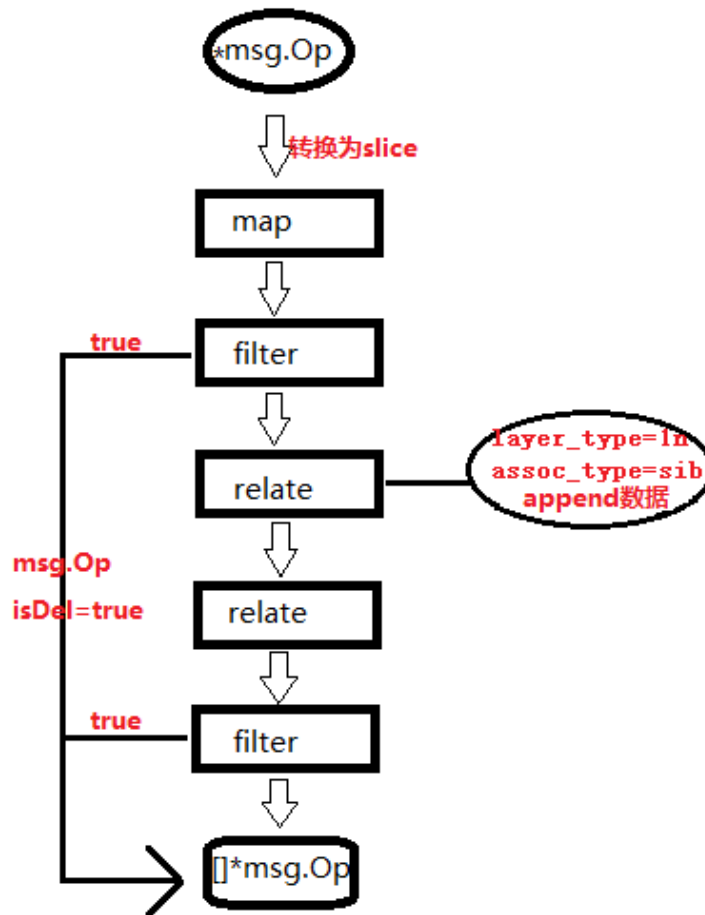


```

        "aimType": "string"
      },
      {
        "srcField": "name",
        "srcType": "string",
        "aimField": "tab_name",
        "aimType": "string"
      }
    ]
  },
  {
    "type": "relateInline",
    "assoc_type": "11",
    "source_id": "no1000001",
    "relate_document_set": "datalink_foo.area_desc",
    "layer_type": "sib",
    "sub_label": "",
    "field_map": {
      "r.id": "area_desc_id"
    },
    "wheres": [
      {
        "src_field": "id",
        "operator": "=",
        "rel_field": "id"
      }
    ]
  },
  {
    "type": "filter",
    "script": "module.exports=function(doc){if(!doc.area_desc_id)
{return false;}return true;}"
  }
]
}

```

对单个文档做有序的处理,pipeline中的item即使执行顺序,如图:



四种flow

该部分提供四种flow: `map`, `filter`, `mapInline`, `relateInline`

- `map`,用于文档字段转换.
- `filter`,返回true就过滤掉文档,不在处理.
- `mapInline`,字段映射.
- `relateInline`,变换关联关系.

需要注意:

1. 执行顺序,pipeline配置中item顺序即为执行顺序,处理过程中向下流转文档.
2. 自定义函数时,需要注意冲突.flow中使用了otto的VM,默认加载了js的underscore库, [underscore的说明](#)

map

对文档的编辑,比如新增字段,删除字段等.

```
{
  "type": "map",
```

```
"script": "module.exports=function(doc){doc.attr1=2022;return doc;}"
}
```

```
module.exports = function (doc) {
  doc.attr1 = 2022;
  return doc;
}
```

注意:删除时,doc文档中可能没有字段

filter

是个过滤文档,返回false为继续处理,true文档则过滤掉文档,不再处理

```
{
  "type": "filter",
  "script": "module.exports=function(doc){return doc.id>100;}"
}
```

```
module.exports = function (doc) {
  return doc.id > 100;
}
```

mapInline

字段映射,使用内部定义的规则,效率比map高.只是简单的字段转换,建议使用.

```
{
  "type": "mapInline",
  "field_map": [
    {
      "srcField": "id",
      "srcType": "long",
      "aimField": "tab_id",
      "aimType": "long"
    },
    {
      "srcField": "code",
      "srcType": "string",
      "aimField": "tab_code",
      "aimType": "string"
    },
    {
      "srcField": "name",
      "srcType": "string",

```

```

        "aimField": "tab_name",
        "aimType": "string"
    }
]
}

```

relateInline

文档的关联关系处理,使用的是建立新连接方式,所以需要新建连接源为empty.

```

{
  "type": "relateInline",
  // 关联关系
  "assoc_type": "11",
  // 一对一
  "source_id": "c888b053-45f3-4c98-b219-8acfe67999d1",
  // 关联资源id
  "script": "",
  // 脚本,暂时无效
  "relate_document_set": "database3.document_set",
  // 关联文档集,需要指定资源id
  "layer_type": "sib",
  // 层级关系:sib sub
  "sub_label": "",
  // 层级关系为sub时,指定key
  "field_map": {
    // 层级关系为sib时,指定字段别名
    // 另外,设定field_map时,没有指定的字段都将被忽略
    // |之后为默认值,当value=nil时,有效
    // i为int,s为string,d为日期now或者时间戳,f为float
    "r.id": "area_desc_id|i:0",
    "s.name": "src_name|s:default_string",
    "s.created_at": "src_name|d:now",
    "s.fund": "src_name|f:0.0"
  },
  "wheres": [
    // 字段关联关系
    {
      "src_field": "id",
      "operator": "=",
      "rel_field": "id"
    }
  ]
}

```

- **source_id**指定数据来源,不设置则为doc读取数据源
- **script**使用脚本进行关联,和一般关联并不共存.暂时不支持
- **assoc_type**定义关联关系,值为: **11(一对一)**/**1n(一对多)**
- **relate_document_set**为关联文档集,在es中为mongodb的 **db.collection**
- **layer_type**为关联数据的层叠关系,值为: **sib(同级)**/**sub(子级)**

- `sub_label`当`layer_type=sub`时,指定子级在当前文档中的key
- `field_map`当`layer_type=sib`时,当前文档字段别名,`r.id`中的`r`指定`relate_document_set`,`s.name`为当前文档中的`name`
- `wheres`为条件关联,默认使用`src_field`与`rel_field`为相等的关系,`operator`暂时无效

Otto虚拟机

注意:

js虚拟机otto不支持诸多浏览器的语法

- 只支持`var`定义变量
- 循环可用 `for in`
- js被压缩之后可能js语法识别错误
- 当心源数据类型中的`bigint`被otto忽略掉精度
- 引入了js函数库`underscore`

虚拟机的内置函数

在flow中使用script时,可直接使用内置函数

- `ifnull` 取默认值
 - `function ifnull(value, default_value) mixed`
- `pad` 拼接字符串
 - `function pad(string,pad_char,number,direction) string`
- `trim` 去掉空白字符
 - `function trim(string) string`
- `startswith` 检查字符串以指定字符开头
 - `function startwith(string, prefix_string) string`
- `endwith` 检查字符串以指定字符结尾
 - `function endwith(string, suffix_string) boolean`
- `strtoupper` 字符串大写
 - `function strtoupper(string) string`
- `strtolower` 字符串小写
 - `function strtolower(string) string`
- `strrev` 字符串翻转
 - `function strrev(string) string`
- `strtotime` 时间转时间戳(10位),
 - `function strtotime(string) integer`
- `date` 格式化时间,支持将10位时间戳转为Y-m-d,Y-m-d H:i:s格式数据
 - `function date('Y-m-d', timestamp) string`
- `now` 当前时间戳10位
 - `function now() integer`
- `ceil` 向上取整
 - `ceil(float) integer`
- `floor` 向下取整
 - `floor(float) integer`
- `round` 四舍五入

- round(float) integer
- rand 随机数
 - rand(float) integer
- abs 绝对值
 - abs(integer) integer
- max 最大值
 - max(array) integer
 - max(value1, value2, ...) integer
- min 最小值
 - min(array) integer
 - min(value1, value2, ...) integer
- DocumentFind 从指定资源中查找唯一文档,目前只支持mysql
 - DocumentFind(id, documentSet, resourceID)